

APRENDIZAJE AUTOMÁTICO

ACTIVIDAD 2 - 3ER CORTE

Diferencia Temporal: SARSA y Q-Learning

Entregable: un notebook .ipynb completamente ejecutado, organizado con celdas de código y celdas de texto para el análisis. El notebook debe ser reproducible por cualquier persona que lo ejecute.

Cada grupo trabajará con un entorno diferente: un grupo trabajará con CliffWalking-v0 y otro grupo con Blackjack-v1.

Nota conceptual: Los algoritmos de Diferencia Temporal (SARSA y Q-Learning) no requieren conocer el modelo del entorno (probabilidades de transición ni función de recompensa explícita), a diferencia de Programación Dinámica. Y a diferencia de Monte Carlo, actualizan la tabla Q tras cada paso individual sin esperar al final del episodio. Esta doble diferencia debe quedar clara en el análisis a lo largo del notebook.

1. Exploración del entorno

Describan el espacio de estados, el espacio de acciones, la función de recompensa y las condiciones de finalización del episodio en su entorno. Para esto deben responder claramente:

- ¿Cuántos estados tiene el entorno y qué representa cada uno?
- ¿Cuántas acciones hay disponibles y qué hace cada una?
- ¿Qué recompensa recibe el agente en cada paso? ¿Cuándo recibe recompensa positiva y cuándo negativa o nula?
- ¿Cuándo termina un episodio?

Ejecuten un episodio completo con política aleatoria y muestren al menos tres pasos con `env.render()` o imprimiendo el estado en cada paso.

Calculen la recompensa promedio y la tasa de éxito de la política aleatoria sobre 1000 episodios. Este valor será la línea base de comparación para toda la actividad.

CliffWalking: Visualicen el mapa del entorno e identifiquen la zona del acantilado, el punto de inicio y el punto de llegada. Muestren qué ocurre en términos de recompensa acumulada cuando el agente cae al acantilado durante un episodio aleatorio.

Blackjack: El estado en Blackjack es una tupla de tres valores: (suma de las cartas del jugador, carta visible del dealer, si el jugador tiene un as utilizable). Expliquen qué significa cada componente con un ejemplo concreto. Indiquen cuántos estados posibles existen en total y por

qué el espacio de estados de Blackjack no puede manejarse igual que el de un entorno de cuadrícula. Además, dado que ya trabajaron Blackjack con Monte Carlo, respondan: ¿esperan que Diferencia Temporal funcione mejor, peor o igual que MC en este entorno? Justifiquen antes de entrenar.

Finalmente, argumenten brevemente por qué en este entorno se puede aplicar Diferencia Temporal y qué ventajas tiene frente a Programación Dinámica y frente a Monte Carlo para este problema específico.}

2. Control con SARSA

Implementen el algoritmo SARSA (on-policy) para entrenar un agente en su entorno. Recuerden que SARSA actualiza la tabla Q usando la acción que realmente va a ejecutar a continuación, elegida con ϵ -greedy. Por eso se llama on-policy: aprende sobre la misma política que usa para explorar.

2.1 Implementación y primer entrenamiento

Implementen la función `sarsa(env, Alpha, gamma, épsilon, K)` que reciba el entorno, los hiperparámetros y el número de episodios K, y retorne la tabla Q aprendida y el historial de retornos por episodio.

Para Blackjack el estado es una tupla, por lo que la tabla Q debe ser un diccionario (defaultdict) en lugar de una matriz numpy. Documenten cómo resolvieron este detalle en una celda de texto.

Usen un set inicial de hiperparámetros arbitrario, documenten cuáles usaron y grafiquen la curva de retorno por episodio con una ventana deslizante de 500 episodios. Expliquen qué observan: ¿el agente mejora con el tiempo?, ¿en qué punto parece estabilizarse?

2.2 Ajuste del hiperparámetro Alpha

Entrenen SARSA con cuatro valores de Alpha: 0.1, 0.3, 0.6 y 0.9, manteniendo los demás hiperparámetros fijos. Grafiquen las cuatro curvas de entrenamiento en una figura de 2x2 subplots con la ventana deslizante de 500 episodios.

Expliquen el efecto de Alpha sobre la velocidad y estabilidad del aprendizaje. ¿Con qué valor converge más rápido?, ¿con cuál es más estable?, ¿hay un valor que haga que el aprendizaje sea errático o que no converja? Elijan el mejor valor y justifiquen la elección.

2.3 Ajuste del hiperparámetro épsilon

Fijando el Alpha elegido, entrenen SARSA con tres valores de épsilon: $\epsilon = 0.05$, $\epsilon = 0.15$ y $\epsilon = 0.30$. Usen el mismo número de episodios K para los tres y repórtenlo claramente. Para cada uno grafiquen la curva de retorno y evalúen la política final sobre 1000 episodios.

épsilon	Alpha usado	Episodios K	Tasa de éxito	Recompensa media
0.05				
0.15				
0.30				

Después de la tabla respondan: ¿con qué valor de ϵ aprende más rápido SARSA?, ¿con cuál obtiene mejor política final?, ¿qué ocurre cuando se explora demasiado o demasiado poco? Elijan el mejor ϵ para SARSA y justifiquen.

2.4 Ajuste del factor de descuento gamma

Usando el mejor Alpha y ϵ encontrados, entrenen SARSA con $\gamma = 0.7$, $\gamma = 0.9$ y $\gamma = 0.99$. Evalúen cada política sobre 1000 episodios.

gamma	epsilon usado	Tasa de éxito	Recompensa media
0.70			
0.90			
0.99			

Expliquen el efecto de gamma: ¿cómo cambia el comportamiento del agente cuando gamma es bajo versus alto en su entorno específico? ¿Tiene sentido que un gamma alto funcione mejor aquí? Conecten la explicación con la naturaleza del entorno descrita en la sección 1.

Blackjack: En Blackjack los episodios son cortos (pocas jugadas) y la recompensa llega casi siempre al final. Teniendo esto en cuenta, ¿esperarían que gamma tenga mucho o poco efecto en este entorno? Contrasten esa expectativa con lo que observaron en los resultados.

3. Control con Q-Learning

Implementen el algoritmo Q-Learning (off-policy). Recuerden que la única diferencia con SARSA está en cómo se elige la acción para calcular el valor futuro en la actualización: Q-Learning siempre usa $\arg\max$ (la mejor acción conocida), aunque en la práctica el agente ejecute una acción epsilon-greedy.

3.1 Comparación directa bajo condiciones idénticas

Implementen la función `qlearning(env, Alpha, gamma, epsilon, K)`. Usen exactamente los mismos hiperparámetros finales que usaron con SARSA y el mismo número de episodios K. Grafiquen en una sola figura las curvas de entrenamiento de SARSA y Q-Learning con ventana de 500 episodios. El objetivo es observar si Q-Learning converge más rápido, más lento o igual que SARSA bajo condiciones idénticas.

3.2 Ajuste de Alpha para Q-Learning

Repitan el ajuste de Alpha (valores 0.1, 0.3, 0.6, 0.9) ahora con Q-Learning, manteniendo los demás hiperparámetros fijos. Grafiquen las cuatro curvas en un subplot 2x2.

¿El mejor Alpha para Q-Learning es el mismo que para SARSA o difieren? Expliquen por qué podría haber diferencia, pensando en cómo cada algoritmo usa esa tasa de aprendizaje en sus actualizaciones.

3.3 Ajuste de epsilon para Q-Learning

Con el mejor Alpha para Q-Learning, repitan el experimento de ϵ con los mismos tres valores (0.05, 0.15, 0.30) y completen la siguiente tabla:

ϵ	Alpha usado	Episodios K	Tasa de éxito	Recompensa media
0.05				
0.15				
0.30				

4. On-policy vs. Off-policy: la diferencia

El objetivo es observar empíricamente la diferencia entre SARSA y Q-Learning que va más allá de la velocidad de convergencia.

CliffWalking: Entrenen ambos algoritmos con $\epsilon = 0.1$ y los mejores Alpha y gamma de cada uno. Durante el entrenamiento (no en la evaluación final), registren qué estados visita el agente con más frecuencia y visualicen las rutas típicas de cada algoritmo sobre el mapa. SARSA tiende a aprender una ruta segura alejada del acantilado. Q-Learning tiende a aprender la ruta más corta, que pasa al borde del acantilado. Muestren evidencia visual de este comportamiento y expliquen por qué ocurre desde la intuición del algoritmo y conectándolo con la fórmula de actualización de cada algoritmo.

Blackjack: Visualicen la política aprendida por SARSA y por Q-Learning como una tabla 2D donde las filas son la suma de cartas del jugador (12 a 21), las columnas son la carta visible del dealer (1 a 10) y el valor de cada celda indica la acción recomendada: pedir carta (P) o plantarse (S). Hagan una tabla para cuando hay as utilizable y otra para cuando no lo hay. En total serán cuatro tablas: dos para SARSA y dos para Q-Learning. Comparen visualmente las políticas aprendidas por ambos algoritmos: ¿son iguales o difieren en algunas situaciones? ¿Cuál se parece más a la estrategia básica de Blackjack (que pueden consultar como referencia)? Expliquen si encuentran diferencias y a qué creen que se deben.

Para ambos grupos: con $\epsilon = 0$ (sin exploración en absoluto), ¿qué diferencia habría entre SARSA y Q-Learning durante el entrenamiento? ¿Y durante la evaluación? Respondan conceptualmente, no es necesario ejecutar este experimento.

5. Comparación SARSA vs. Q-Learning

Usen los mejores hiperparámetros encontrados para cada algoritmo por separado (no tienen que ser los mismos). Entrenen ambos agentes y evalúen cada política final sobre 1000 episodios.

Algoritmo	Alpha	Épsilon	Gamma	Episodios K	Tasa de éxito	Recompensa media
SARSA						
Q-Learning						

Grafiquen en una sola figura las curvas de entrenamiento del mejor SARSA y el mejor Q-Learning con sus respectivos mejores hiperparámetros. Usen ventana de 500 episodios y leyenda clara.

Luego respondan las siguientes preguntas, cada una con al menos un párrafo de justificación:

- ¿Cuál algoritmo logró mejor desempeño final en su entorno?
- ¿Cuál convergió más rápido? ¿En qué iteración aproximada se estabilizó cada uno?
- ¿Cuál fue más sensible al valor de Alpha? ¿Por qué creen que ocurre eso?
- ¿En qué tipo de problema real preferiría usar SARSA sobre Q-Learning, y al revés? Piensen en situaciones donde el riesgo durante el entrenamiento importa.
- ¿Qué ventajas tienen estos algoritmos frente a Monte Carlo? ¿Y frente a Programación Dinámica?

6. Conexión con Programación Dinámica y Monte Carlo

Esta parte cierra el hilo de las tres actividades del corte.

CliffWalking: CliffWalking no fue usado en las actividades anteriores, así que usen la línea base de la política aleatoria (parte 1) como referencia inferior. Si desean ir más allá, pueden implementar Iteración de Valores para CliffWalking (el modelo es conocido) y usar ese resultado como referencia superior. No es obligatorio, pero suma al análisis.

Blackjack: Blackjack fue el entorno central de la actividad de Monte Carlo. Recuperen la tasa de éxito y la recompensa media obtenida con el mejor método MC de esa actividad para usarla como referencia de comparación.

Grafiquen la curva de desempeño (recompensa media por episodio con ventana de 500) del mejor SARSA y el mejor Q-Learning. Tracen sobre ese gráfico líneas horizontales punteadas con los niveles de referencia disponibles (política aleatoria, MC o PD según corresponda). Esto permite observar cuántos episodios necesita Diferencia Temporal para alcanzar esos niveles de desempeño.

Completen la siguiente tabla resumen comparando los paradigmas vistos en el corte (completen solo las columnas que apliquen a su entorno):

Criterio	PD	Monte Carlo	SARSA	Q-Learning
Necesita modelo del entorno	Sí	No	No	No
Aprende de experiencia	No	Sí	Sí	Sí
Actualiza al final del episodio	No	Sí	No	No
Actualiza en cada paso	Sí	No	Sí	Sí
On-policy / Off-policy	N/A	On	On	Off
Tasa de éxito en su entorno				
Recompensa media final				

Finalicen con un párrafo de conclusión que responda: dado lo observado en las tres actividades, ¿en qué tipo de problema real elegiría usar cada uno de estos cuatro enfoques y por qué?

Notas sobre entregables y reproducibilidad

- El notebook debe ejecutarse de arriba a abajo sin errores, con todas las celdas ya ejecutadas.
- Cada bloque de código debe estar acompañado de una celda de texto que explique qué hace y por qué. No es suficiente con comentarios dentro del código.
- Todas las gráficas deben tener títulos, etiquetas en los ejes y leyenda cuando corresponda.
- Las tablas deben completarse con los valores reales obtenidos en la ejecución, no con valores estimados.
- El análisis escrito debe ser propio: no es suficiente describir lo que se ve en las gráficas, hay que explicar por qué conectándolo con la teoría vista en clase.

Cómo cargar cada entorno

CliffWalking-v0: `env = gym.make('CliffWalking-v0')`. Cuadrícula 4×12 (48 estados), 4 acciones (arriba, abajo, izquierda, derecha). Recompensa de -1 por cada paso y de -100 al caer al acantilado. El episodio termina al llegar a la meta o al caer al acantilado.

Blackjack-v1: `env = gym.make('Blackjack-v1', natural=False, sab=False)`. El estado es una tupla (suma del jugador: 4-21, carta visible del dealer: 1-10, as utilizable: True/False). 2 acciones: 0 = plantarse, 1 = pedir carta. Recompensa de +1 al ganar, -1 al perder, 0 al empatar. La tabla Q debe implementarse como un diccionario indexado por la tupla de estado.